Autonomous Legal Researcher Agent

Course Name: Agentic AI

Institute Name : Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Jogendar Das Bairagi	EN22ME304045
2.	Roshan Bankar	EN22EL301072
3.	Vivek Gangrade	EN22CS3011098
4.	Maahi Maheshwari	EN22ME304057
5.	Shubhangi Prajapat	EN22ME304094

Group Name : 04D10

Project Number : DO-17

Industry Mentor Name : Mr. Ashwin Krishnan

University Mentor Name : Prof Maya Yadav Baniya

Academic Year : 2025-26

Table of Contents

1. Problem Statement & Objectives

1.1 Problem Statement

1.2 Project Objectives

1.3 Scope of the Project

2. Proposed Solution

2.1 Key Features

2.2 Overall Architecture / Workflow

2.3 Tools & Technologies Used

3. Results & Output

3.1 Screenshots / Output

3.2 Data Model

3.3 Key Outcomes

4. Conclusion

5. Future Scope & Enhancements

6. References

1. Problem Statement & Objectives

1.1 Problem Statement

Legal professionals, researchers, and students routinely spend considerable time manually reviewing dense, multi-page legal documents, statutes, and case law to extract specific provisions, precedents, or definitions. This manual process is time-consuming, error-prone, and highly inaccessible to non-experts seeking legal guidance.

There is a pressing need for an intelligent, autonomous system that can understand natural language queries, retrieve real-time legal data from authoritative sources, and generate accurate, structured legal reports — all without requiring the user to have any domain expertise.

1.2 Project Objectives

- Develop an AI-driven autonomous agent capable of answering legal queries in natural language.
- Integrate a real-time web search pipeline (Tavily API) to retrieve up-to-date legal information from authoritative sources.
- Leverage a Large Language Model (Groq LLM) to generate structured, human-readable legal explanations.
- Implement a domain validation layer to restrict responses strictly to legal topics, preventing off-domain outputs.
- Build a user-friendly Gradio web interface with example queries and downloadable PDF reports.
- Ensure system security through environment variable-based API key management and HTTPS communication.
- Design a stateless, privacy-preserving architecture where no user data is permanently stored.

1.3 Scope of the Project

This project covers the end-to-end design and implementation of the Autonomous Legal Researcher Agent. The scope includes:

- User query intake and domain-level validation using keyword-based filtering.
- Real-time legal data retrieval via the Tavily Search API.
- Context aggregation from multiple web sources into a unified prompt.
- Structured legal report generation via the Groq LLM API (Llama 3 / Mixtral).
- Output formatting and downloadable PDF report generation using FPDF.
- A Gradio-based conversational web interface with example prompts and file download capability.

2. Proposed Solution

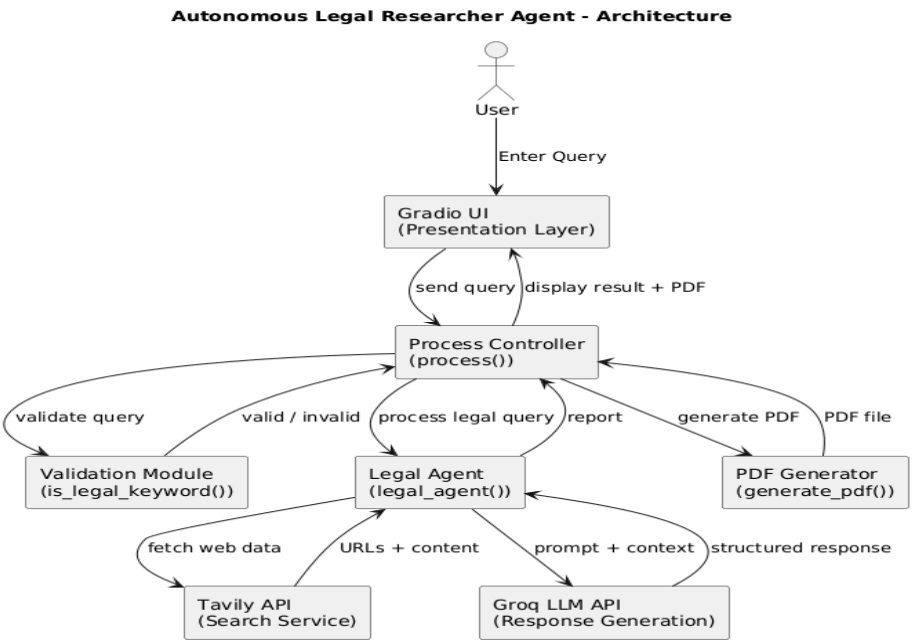
2.1 Key Features

- Natural Language Legal Q&A: Users submit plain-English legal questions and receive clear, section-structured answers grounded in real-time retrieved data.
- Domain Validation Guard: The is_legal_keyword() function filters out non-legal queries before any processing occurs, ensuring domain accuracy.
- Real-Time Web Retrieval: The Tavily API performs live searches and returns structured results including URLs and content snippets from authoritative legal sources.
- LLM-Powered Structured Reports: Groq's fast inference API generates legally organized reports with sections for Clear Explanation, Key Laws, Important Points, and Conclusion.
- Automated PDF Report Generation: Using FPDF, each response is formatted into a professional, downloadable PDF report with proper headings and bullet formatting.
- Privacy-First Stateless Design: No user queries or API responses are stored permanently — all data is processed in-memory and discarded after each session.
- Lightweight Gradio Interface: The web UI provides query input, example prompts, real-time output display, and a one-click PDF download.
-

2.2 Overall Architecture / Workflow

The system follows a modular, layered architecture with clear separation of concerns across seven distinct functional layers:

Architecture Diagram



Three-Tier System Architecture

- **Presentation Tier (Gradio):** Web-based frontend for user query input, output rendering, and PDF download. Maintains stateless interaction per request.
- **Intelligence Tier (legal_ agent + Groq LLM):** Orchestrates the full pipeline — validation, search coordination, context building, and LLM inference. The Groq-powered LLM acts as the Legal Expert layer.
- **Data Retrieval Tier (Tavily API):** Performs real-time internet searches and returns structured results that are aggregated into a contextual prompt for the LLM.

2.3 Tools & Technologies Used

Technology	Role	Details
Python	Backend & Orchestration	Primary language for all modules
Gradio	Web UI Framework	Rapid chatbot/form interface
Tavily API	Real-Time Web Search	Fetches live legal data as JSON
Groq API	LLM Inference	Fast LPU-accelerated text generation
FPDF	PDF Report Generation	Converts structured text to PDF
LangChain	Orchestration (optional)	Agent workflow management
Python dotenv	Security	Environment variable API key management

3. Results & Output

3.1 Screenshots / Outputs

The following section describes the working Autonomous Legal Researcher Agent interface and its key interaction flows.

Figure 1: Gradio Interface — Query Input Panel

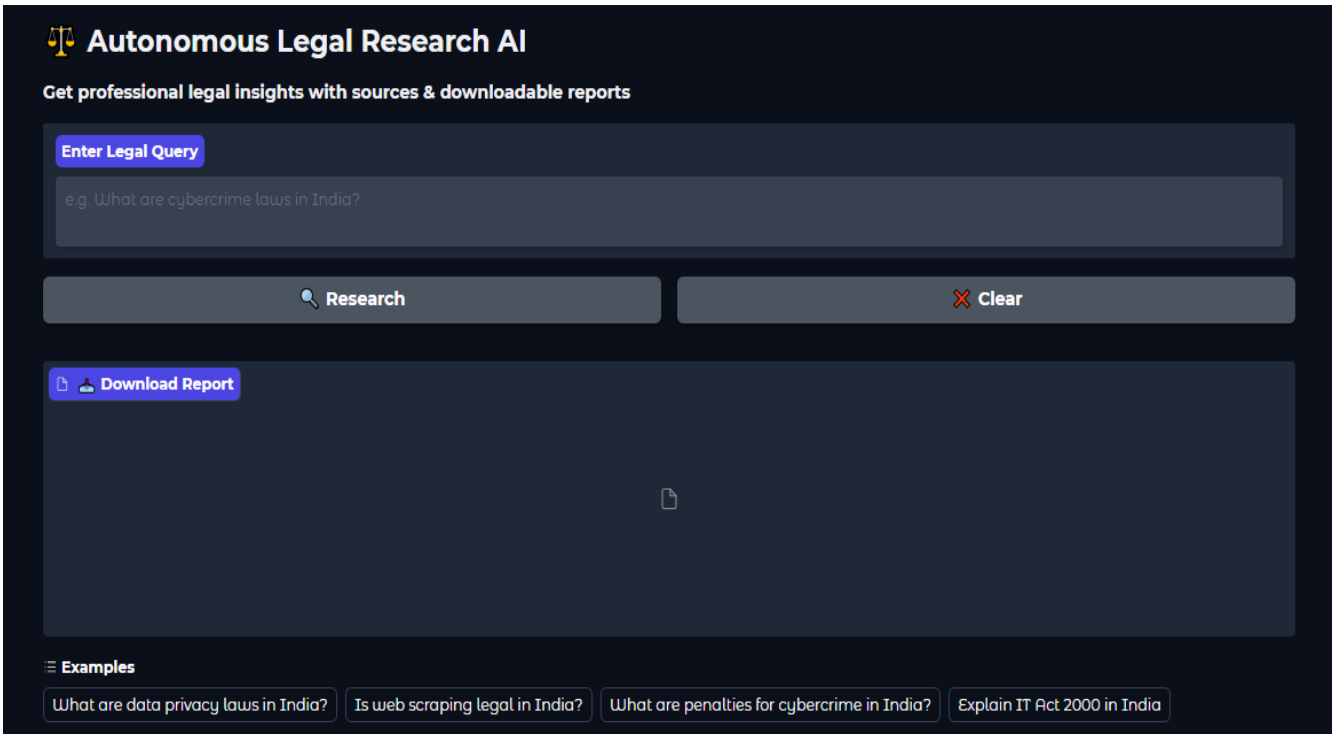
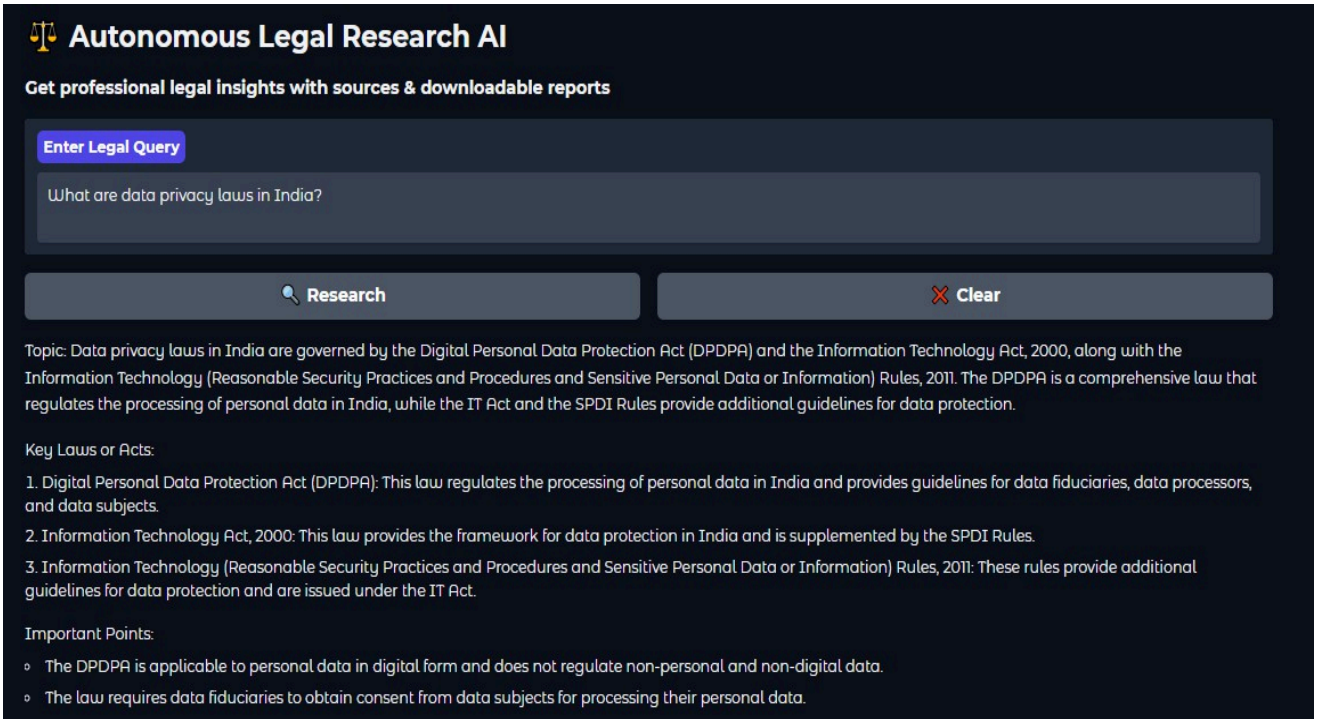
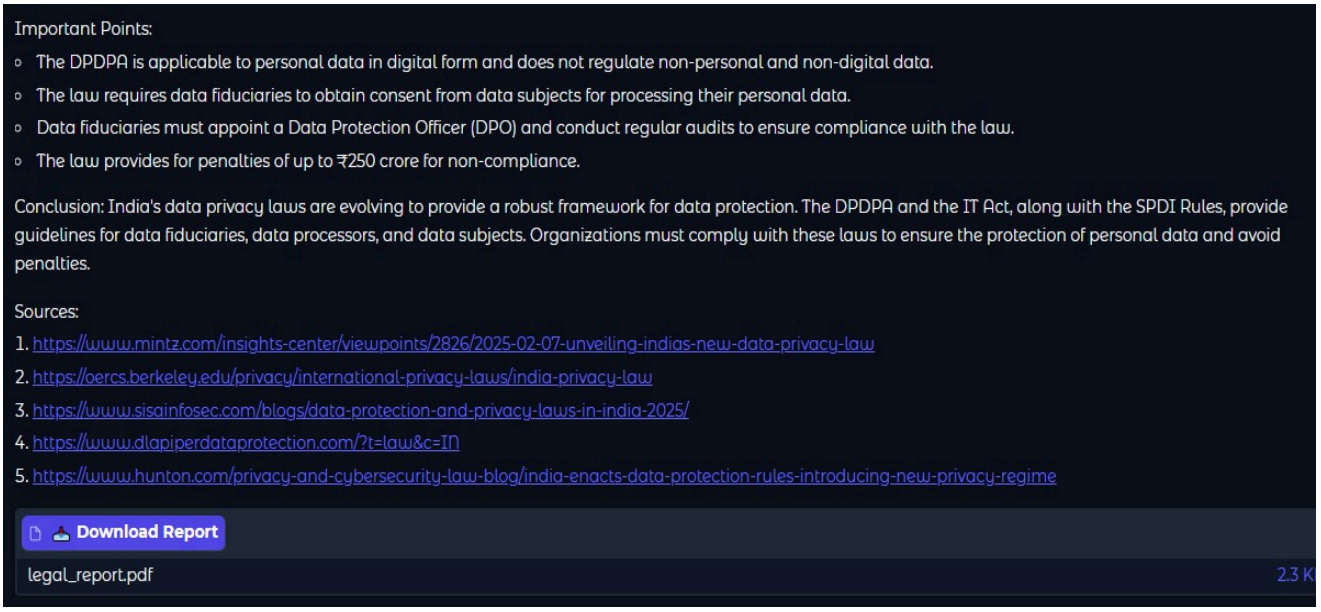


Figure 2: Sample Query — Legal Research Output





3.2 Data Model & Pipeline

The diagram below describes the data transformation pipeline from raw user input to a formatted, downloadable legal report.

Raw Query (String)
↓
Validation Layer → LEGAL / INVALID
↓ (valid path)
Tavily API Search → JSON (URLs + Snippets)
↓
Context Builder → Unified Context String
↓
Groq LLM Layer → Structured Legal Report
↓
Formatter → Clean Text (headings standardized)
↓
FPDF Generator → PDF File
↓
Gradio UI → Display + Download Option

3.3 Key Outcomes

- Successfully built a fully functional Autonomous Legal Researcher Agent using a modular agentic architecture.
- Achieved reliable real-time legal data retrieval from authoritative sources via the Tavily Search API.
- Demonstrated accurate, structured report generation with domain-restricted LLM responses through the Groq API.
- Zero data persistence validated — all query data and API responses are processed in-memory only.
- Domain validation layer confirmed effective — non-legal queries are consistently rejected with user-friendly error messages.
- PDF report generation working end-to-end: formatted reports downloadable directly from the Gradio interface.
- Stateless architecture ensures user privacy and eliminates long-term data depen

4. Conclusion

The Autonomous Legal Researcher Agent demonstrates how Agentic AI can be effectively applied in the legal domain. By combining domain-restricted validation, real-time retrieval (Tavily API), and fast LLM inference (Groq API), the system generates structured and reliable legal reports from natural language queries.
A key learning is the importance of domain guardrails, which ensure only relevant legal queries are processed, reducing incorrect outputs. The modular architecture—covering validation, retrieval, processing, and report generation—improves scalability and maintainability.
Overall, the project provides practical experience in agentic workflows, API integration, and prompt engineering, successfully connecting theoretical AI concepts with real-world implementation

5. Future Scope & Enhancements

The current system serves as a strong proof-of-concept and can be further enhanced into a production-ready platform. Future improvements include expanding to multiple domains, integrating Retrieval-Augmented Generation (RAG) with vector databases, and enabling persistent storage for user history.
Additional features such as user authentication, advanced legal citation verification, and fine-tuned domain-specific LLMs can improve reliability and personalization. Enhancements in UI like source linking, multilingual support, caching mechanisms, and a mobile application will further increase accessibility, efficiency, and overall user experience.

6. References

1. Tavily AI Search API Documentation. https://docs.tavily.com
2. Groq API Documentation & LPU Architecture. https://console.groq.com/docs
3. FPDF Python Library Documentation. https://pyfpdf.readthedocs.io
4. Gradio Documentation — Build & Share Machine Learning Apps. https://www.gradio.app/docs
5. LangChain Documentation — Agent & RAG Pipeline Reference. https://docs.langchain.com.
6. OpenAI / Meta LLaMA Research Papers — Large Language Models for Domain-Specific Tasks.